

LLM & GEN AI

Projet 1 : ESILV Smart Assistant Multi-Agent Chatbot

1. Introduction générale

1.1 Contexte du projet

Dans un contexte de transformation numérique croissante, les établissements d'enseignement supérieur sont amenés à repenser leurs outils de communication et d'information afin de répondre efficacement aux attentes des étudiants, des candidats et des partenaires institutionnels. L'École Supérieure d'Ingénieurs Léonard de Vinci (ESILV) propose une offre de formation riche et diversifiée, couvrant plusieurs cycles et spécialités, ce qui engendre un volume important d'informations à destination du public.

Actuellement, ces informations sont principalement accessibles via des supports statiques tels que des sites web, des brochures ou des documents PDF. Bien que complets, ces supports présentent certaines limites : navigation parfois complexe, difficulté à obtenir des réponses précises à des questions spécifiques, et sollicitation accrue des services administratifs pour des demandes récurrentes. Dans ce contexte, l'utilisation d'un assistant conversationnel intelligent apparaît comme une solution pertinente pour améliorer l'accessibilité et la diffusion de l'information.

1.2 Problématique

La mise en place d'un chatbot éducatif soulève plusieurs défis techniques et fonctionnels. Un assistant conversationnel doit être capable de fournir des réponses fiables, à jour et contextualisées, tout en conservant une interaction naturelle avec l'utilisateur. Les modèles de langage de grande taille (LLM) offrent des capacités avancées de compréhension et de génération de texte, mais présentent des limites importantes, notamment le risque d'hallucinations et l'absence de connaissances spécifiques ou actualisées sur une institution donnée.

Par ailleurs, certaines interactions dépassent le simple cadre de la réponse informative. Les utilisateurs peuvent souhaiter effectuer des actions structurées, telles que demander une brochure, planifier un contact ou laisser leurs coordonnées pour un suivi. La problématique centrale du projet est donc la suivante : comment concevoir un assistant conversationnel capable de combiner des réponses factuelles précises, basées sur des documents institutionnels, et des interactions structurées nécessitant une gestion fine du dialogue ?

1.3 Objectifs du projet

L'objectif principal de ce projet est de concevoir et d'implémenter un assistant intelligent dédié à l'ESILV, capable de répondre aux besoins informationnels et interactionnels des utilisateurs. Plus précisément, le projet vise à :

- ❖ Mettre en place un chatbot capable de répondre aux questions relatives aux formations, aux admissions et aux cours proposés par l'ESILV.
- ❖ Exploiter une approche de Retrieval-Augmented Generation (RAG) afin de garantir des réponses fondées sur des sources documentaires fiables.
- ❖ Concevoir une architecture multi-agents, dans laquelle chaque agent est spécialisé dans une tâche précise (classification d'intention, recherche documentaire, génération de réponses, collecte de données).
- ❖ Permettre la collecte structurée d'informations utilisateur (coordonnées, demandes spécifiques) à travers une interaction conversationnelle fluide.
- ❖ Proposer une interface utilisateur intuitive, accessible via une application web.

1.4 Organisation du rapport

Ce rapport est structuré de manière à présenter de façon progressive les différentes dimensions du projet. La section suivante introduit les concepts fondamentaux et l'état de l'art relatifs aux chatbots, aux modèles de langage et aux architectures multi-agents. La troisième partie décrit l'architecture globale du système et le rôle de chaque composant. Les sections suivantes détaillent l'orchestration du workflow ainsi que le fonctionnement des agents principaux. Enfin, les dernières parties sont consacrées à l'interface utilisateur, à l'évaluation du système, à ses limites et aux perspectives d'amélioration.

2. État de l'art et concepts clés

2.1 Chatbots et modèles de langage

Les chatbots modernes reposent majoritairement sur des modèles de langage de grande taille (LLM), capables de comprendre et de générer du texte en langage naturel. Ces modèles offrent une grande flexibilité et permettent de traiter des requêtes variées sans règles prédéfinies. Cependant, leurs connaissances sont limitées à leur phase d'entraînement et ils peuvent produire des réponses incorrectes ou non vérifiables, ce qui pose un problème dans un contexte institutionnel nécessitant des informations fiables.

2.2 Retrieval-Augmented Generation (RAG)

La Retrieval-Augmented Generation (RAG) vise à combiner les capacités génératives des LLM avec un système de recherche documentaire. Lorsqu'une requête est formulée, des documents pertinents sont récupérés depuis une base externe et utilisés comme contexte pour générer la réponse. Cette approche réduit les hallucinations et permet d'ancrer les réponses sur des sources institutionnelles actualisables, ce qui est particulièrement adapté aux cas d'usage éducatifs.

2.3 Architectures multi-agents

Une architecture multi-agents repose sur la collaboration de plusieurs agents spécialisés, chacun responsable d'une tâche précise. Cette approche améliore la modularité et la maintenabilité du système. Dans un assistant conversationnel, elle permet de distinguer clairement la classification d'intention, la recherche d'information, la génération de réponses et la gestion d'interactions structurées, tout en assurant une orchestration cohérente du dialogue.

3. Architecture générale du système

3.1 Vue d'ensemble

Le projet ESILV Smart Assistant repose sur une architecture modulaire combinant une interface utilisateur, un orchestrateur central et plusieurs agents spécialisés. Cette architecture vise à séparer clairement les responsabilités fonctionnelles tout en assurant une coordination fluide entre les différents composants du système.

L'utilisateur interagit avec le chatbot via une interface web développée avec Streamlit. Les requêtes sont ensuite prises en charge par un orchestrateur qui analyse l'intention de l'utilisateur et délègue le

traitement à l'agent le plus approprié. Les réponses sont générées soit à partir de documents institutionnels grâce à un système RAG, soit via des interactions conversationnelles structurées.

3.2 Description des composants

L'architecture s'articule autour des composants suivants :

- ❖ Interface utilisateur (Streamlit)
Permet la saisie des requêtes, l'affichage des réponses, l'upload de documents et l'accès à un tableau de bord administrateur.
- ❖ Orchestrateur (Workflow Manager)
Joue un rôle central en coordonnant les échanges entre les agents. Il reçoit la requête utilisateur, déclenche la classification d'intention et redirige la requête vers le pipeline approprié.
- ❖ RouterAgent
Responsable de la classification de l'intention utilisateur (informationnelle, conversationnelle ou action structurée).
- ❖ RetrieverAgent
Effectue une recherche sémantique dans une base vectorielle afin de récupérer les documents les plus pertinents.
- ❖ QAGeneratorAgent
Génère une réponse à partir de la requête utilisateur et des documents récupérés.
- ❖ FormFillerAgent
Gère les interactions multi-tours nécessaires à la collecte d'informations utilisateur (coordonnées, demandes spécifiques).
- ❖ Base de connaissances
Constituée de documents ESILV préalablement indexés dans une base vectorielle.

3.3 Flux de traitement d'une requête

Le traitement d'une requête suit un flux logique bien défini :

1. L'utilisateur soumet une requête via l'interface Streamlit.
2. L'orchestrateur transmet la requête au RouterAgent pour identifier l'intention.
3. Selon l'intention détectée :
 - Pour une demande informationnelle, la requête est transmise au RetrieverAgent, puis au QAGeneratorAgent.
 - Pour une interaction structurée, la requête est prise en charge par le FormFillerAgent.

La réponse finale est renvoyée à l'interface utilisateur.

Cette séparation des flux permet d'adapter dynamiquement le comportement du chatbot en fonction du besoin réel de l'utilisateur.

3.4 Justification des choix architecturaux

Le choix d'une architecture multi-agents permet d'améliorer la lisibilité, la maintenabilité et l'évolutivité du système. Chaque agent étant spécialisé, il est possible d'ajouter de nouvelles

fonctionnalités ou d'améliorer un composant sans impacter l'ensemble du système. Cette approche est particulièrement adaptée aux systèmes conversationnels complexes intégrant à la fois des réponses factuelles et des interactions structurées.

4. Orchestration et gestion du workflow

4.1 Rôle de l'orchestrateur

L'orchestrateur constitue le cœur du système ESILV Smart Assistant. Il est responsable de la coordination entre les différents agents et garantit la cohérence globale du dialogue. Son rôle principal est de centraliser la logique décisionnelle, tout en maintenant une séparation claire des responsabilités entre les agents spécialisés.

Plutôt que de laisser chaque agent interagir directement avec l'utilisateur, l'orchestrateur agit comme un point de contrôle unique. Cette approche permet de maîtriser le flux de traitement, de faciliter le débogage et d'assurer une meilleure évolutivité du système.

4.2 Logique de traitement d'une requête

Lorsqu'une requête utilisateur est reçue, l'orchestrateur déclenche en premier lieu une phase de classification d'intention. Cette étape permet d'identifier la nature de la demande et d'orienter le traitement vers le pipeline approprié.

La logique décisionnelle repose sur une distinction simple :

- ❖ Les requêtes informationnelles sont redirigées vers le pipeline RAG, combinant recherche documentaire et génération de réponse.
- ❖ Les requêtes nécessitant une action structurée sont prises en charge par l'agent dédié à la collecte d'informations.

Cette logique conditionnelle permet d'adapter dynamiquement le comportement du chatbot sans complexifier les agents eux-mêmes.

4.3 Implémentation du workflow

L'implémentation du workflow repose sur une fonction centrale qui enchaîne les appels aux agents selon l'intention détectée. Cette fonction agit comme un routeur logique, garantissant que chaque requête suit un parcours de traitement cohérent.

Le choix d'une orchestration centralisée simplifie l'ajout de nouveaux agents ou de nouveaux types d'intentions. Il devient ainsi possible d'étendre le système sans remettre en cause l'architecture existante, ce qui constitue un avantage majeur dans un projet évolutif.

4.4 Avantages de l'approche adoptée

Cette gestion du workflow présente plusieurs bénéfices :

- ❖ Clarté architecturale : chaque agent remplit un rôle précis.
- ❖ Maintenabilité : les modifications sont localisées et maîtrisées.
- ❖ Scalabilité : de nouveaux agents peuvent être intégrés facilement.
- ❖ Robustesse : le contrôle central réduit les comportements imprévus.

L'orchestrateur joue ainsi un rôle clé dans la fiabilité et la cohérence du système conversationnel.

5. RouterAgent : classification d'intention

5.1 Rôle du RouterAgent

Le RouterAgent est responsable de l'identification de l'intention principale associée à chaque requête utilisateur. Cette étape est essentielle, car elle conditionne l'ensemble du flux de traitement du système. Une mauvaise classification entraînerait un routage inadapté, pouvant dégrader la pertinence de la réponse ou interrompre l'interaction.

Contrairement à une approche basée sur des règles fixes, le RouterAgent s'appuie sur un modèle de langage afin de gérer la diversité et l'ambiguïté des formulations naturelles.

5.2 Intents pris en charge

Le système supporte plusieurs catégories d'intentions couvrant les principaux cas d'usage du chatbot :

- ❖ `program_info` : questions relatives aux formations et parcours proposés.
- ❖ `admission_help` : informations sur les modalités d'admission et d'inscription.
- ❖ `course_details` : détails sur les cours ou modules spécifiques.
- ❖ `general_info` : informations générales sur l'établissement.
- ❖ `contact_collection` : demandes impliquant la collecte de coordonnées ou un suivi.
- ❖ `small_talk` : interactions conversationnelles simples.
- ❖ `unknown` : requêtes non classifiables.

Cette catégorisation permet de distinguer clairement les requêtes nécessitant un traitement informationnel de celles impliquant une interaction structurée.

5.3 Principe de fonctionnement

Le RouterAgent utilise un LLM configuré pour effectuer une classification supervisée à partir d'un prompt spécifique. La requête utilisateur est analysée et associée à une intention unique parmi la liste prédéfinie. Le résultat de cette classification est ensuite transmis à l'orchestrateur, qui déclenche le pipeline de traitement correspondant.

L'utilisation d'un modèle de langage pour cette tâche permet de mieux gérer les formulations complexes ou imprécises, tout en évitant la rigidité des systèmes à base de règles.

5.4 Robustesse et limites

Cette approche offre une bonne robustesse face à la variabilité linguistique des utilisateurs. Toutefois, certaines requêtes ambiguës peuvent conduire à des classifications imprécises, notamment lorsqu'elles combinent plusieurs intentions. Dans ces cas, le système privilégie l'intention dominante ou bascule vers une catégorie générique afin de préserver la continuité de l'interaction.

6. Système RAG : recherche et génération de réponses

6.1 Pipeline d'ingestion des données

Le système RAG repose sur une base de connaissances constituée de documents institutionnels de l'ESILV. Ces documents sont d'abord extraits depuis des fichiers PDF, puis découpés en segments textuels appelés *chunks*. Ce découpage permet d'améliorer la granularité de la recherche et d'éviter l'injection de contextes trop larges lors de la génération des réponses.

Chaque chunk est ensuite transformé en vecteur à l'aide d'un modèle d'embeddings et stocké dans une base vectorielle. Cette phase d'ingestion est réalisée en amont et ne nécessite pas d'être rejouée lors de chaque interaction utilisateur.

6.2 RetrieverAgent : recherche sémantique

Le RetrieverAgent est chargé d'identifier les segments de documents les plus pertinents en fonction de la requête utilisateur. Contrairement à une recherche par mots-clés, la recherche sémantique permet de comparer la similarité de sens entre la requête et les documents indexés.

Pour chaque requête, le RetrieverAgent :

- ❖ génère un embedding de la question,
- ❖ interroge la base vectorielle,
- ❖ sélectionne les k chunks les plus similaires.

Ces documents constituent le contexte informationnel transmis à l'agent de génération. Le paramètre *top-k* permet de trouver un compromis entre richesse du contexte et concision de la réponse.

6.3 QAGeneratorAgent : génération de réponse

Le QAGeneratorAgent est responsable de la production de la réponse finale. Il s'appuie sur un modèle de langage auquel sont fournis :

- ❖ la question utilisateur,
- ❖ les documents récupérés par le RetrieverAgent.

La génération est donc contrainte par le contexte documentaire, ce qui permet de limiter les hallucinations et d'améliorer la précision des réponses. Les paramètres de génération (température, longueur maximale) sont réglés afin de privilégier des réponses factuelles et déterministes, adaptées à un usage institutionnel.

6.4 Apports du RAG dans le projet

L'intégration d'un système RAG permet de combiner la flexibilité des modèles de langage avec la fiabilité de sources documentaires contrôlées. Cette approche est particulièrement adaptée au contexte du projet, où les réponses doivent être à la fois naturelles et rigoureusement alignées avec les informations officielles de l'ESILV.

7. FormFillerAgent : interaction structurée

7.1 Objectif du FormFillerAgent

Le FormFillerAgent est conçu pour gérer les interactions nécessitant la collecte d'informations utilisateur, telles que les demandes de brochures, de rendez-vous ou de prise de contact. Contrairement aux requêtes purement informationnelles, ces interactions impliquent un dialogue structuré sur plusieurs tours et requièrent une validation rigoureuse des données collectées.

7.2 Dialogue multi-tours

Le fonctionnement du FormFillerAgent repose sur un échange progressif avec l'utilisateur. Les informations sont demandées de manière séquentielle, en fonction des données déjà fournies. Cette approche permet de conserver une interaction naturelle tout en garantissant l'exhaustivité des informations requises.

À chaque tour, l'agent analyse la réponse de l'utilisateur, met à jour l'état de la conversation et détermine la prochaine information à collecter. Ce mécanisme permet de gérer des dialogues partiels ou interrompus sans perte de cohérence.

7.3 Validation des données

Afin d'assurer la fiabilité des informations collectées, le FormFillerAgent intègre des mécanismes de validation. Les champs sensibles, tels que l'adresse électronique ou le numéro de téléphone, sont vérifiés à l'aide de règles de format. En cas d'erreur, l'utilisateur est invité à corriger sa saisie, ce qui améliore la qualité des données enregistrées.

7.4 Stockage et exploitation

Une fois l'ensemble des informations collectées et validées, les données sont enregistrées de manière structurée dans un espace de stockage dédié. Elles peuvent ensuite être exploitées via une interface administrative pour des actions de suivi ou de communication. Cette séparation entre la collecte et l'exploitation des données permet de maintenir une architecture claire et sécurisée.

8. Interface utilisateur et expérience

8.1 Choix de l'interface Streamlit

L'interface utilisateur du ESILV Smart Assistant a été développée à l'aide de Streamlit. Ce framework permet de concevoir rapidement des applications web interactives tout en restant intégré à l'écosystème Python. Il est particulièrement adapté aux projets de prototypage et de démonstration de systèmes basés sur l'intelligence artificielle.

8.2 Fonctionnalités principales

L'interface propose un espace de discussion permettant à l'utilisateur de dialoguer directement avec le chatbot. Les réponses sont affichées de manière claire, accompagnées d'informations complémentaires telles que l'intention détectée ou les sources utilisées dans le cas des réponses issues du système RAG.

L'application permet également l'upload de documents et l'accès à un tableau de bord administratif. Ce dernier offre une visualisation synthétique des données collectées, facilitant leur exploitation pour des actions de suivi ou de communication.

8.3 Expérience utilisateur

L'accent a été mis sur la simplicité et la lisibilité de l'interface afin de garantir une prise en main rapide. Le choix d'une interaction conversationnelle permet de réduire la complexité perçue par l'utilisateur, tout en offrant un accès direct aux informations essentielles. Cette approche contribue à améliorer l'expérience utilisateur globale et l'efficacité du système.

9. Évaluation, limites et perspectives

9.1 Évaluation du système

L'évaluation du ESILV Smart Assistant a été réalisée de manière qualitative et fonctionnelle, en se concentrant sur la pertinence des réponses, la robustesse de la classification d'intention et les temps de réponse observés. Les tests ont montré que le système est capable de répondre de manière cohérente à la majorité des requêtes informationnelles, en s'appuyant sur les documents institutionnels fournis.

Le système RAG permet de limiter efficacement les hallucinations du modèle de langage, tandis que l'architecture multi-agents garantit une séparation claire des responsabilités. La classification d'intention s'est révélée globalement fiable pour les cas d'usage courants, assurant un routage correct des requêtes.

9.2 Limites identifiées

Malgré ses performances satisfaisantes, le système présente certaines limites. Les temps de réponse peuvent varier en fonction de la charge et du modèle utilisé, notamment lors des requêtes nécessitant une recherche documentaire. De plus, certaines requêtes ambiguës peuvent être associées à une intention incorrecte, ce qui impacte le flux de traitement.

Par ailleurs, la base de connaissances repose sur un ensemble de documents statiques. Toute mise à jour de ces documents nécessite une réindexation préalable, ce qui peut représenter une contrainte en cas d'évolution fréquente des contenus.

9.3 Perspectives d'amélioration

Plusieurs pistes d'amélioration peuvent être envisagées. L'intégration d'une mémoire utilisateur permettrait de personnaliser davantage les interactions. L'optimisation des modèles ou l'utilisation de versions plus légères pourrait réduire la latence. Enfin, le déploiement sur une infrastructure cloud offrirait une meilleure scalabilité et faciliterait l'accès au service à un plus grand nombre d'utilisateurs.

10. Conclusion générale

Ce projet avait pour objectif de concevoir un assistant conversationnel intelligent dédié à l'ESILV, capable de répondre de manière fiable aux besoins informationnels des utilisateurs tout en gérant des interactions structurées. À travers la mise en œuvre d'une architecture multi-agents combinée à une approche de Retrieval-Augmented Generation, le système développé parvient à concilier flexibilité conversationnelle et rigueur informationnelle.

L'utilisation d'agents spécialisés a permis de structurer clairement le traitement des requêtes, depuis la classification de l'intention jusqu'à la génération de réponses ou la collecte de données. Cette organisation modulaire constitue un atout majeur en termes de maintenabilité et d'évolutivité, tout en facilitant l'intégration de nouvelles fonctionnalités. Le recours au RAG s'est révélé particulièrement pertinent dans un contexte institutionnel, en garantissant des réponses fondées sur des sources documentaires contrôlées.

Au-delà de l'aspect fonctionnel, ce projet a permis de mobiliser et de consolider des compétences transversales en intelligence artificielle, en ingénierie logicielle et en conception de systèmes interactifs. Les limites identifiées ouvrent la voie à de nombreuses améliorations, notamment en matière de personnalisation, de performance et de déploiement à grande échelle. Ainsi, le ESILV Smart Assistant constitue une base solide pour le développement futur d'outils numériques au service de l'enseignement supérieur.

11. Source Code Repository

<https://github.com/AlexandreFrancony/ESILV-Bot>